In [1]:
```
import matplotlib
```

In [2]:
```
import matplotlib.pyplot
```

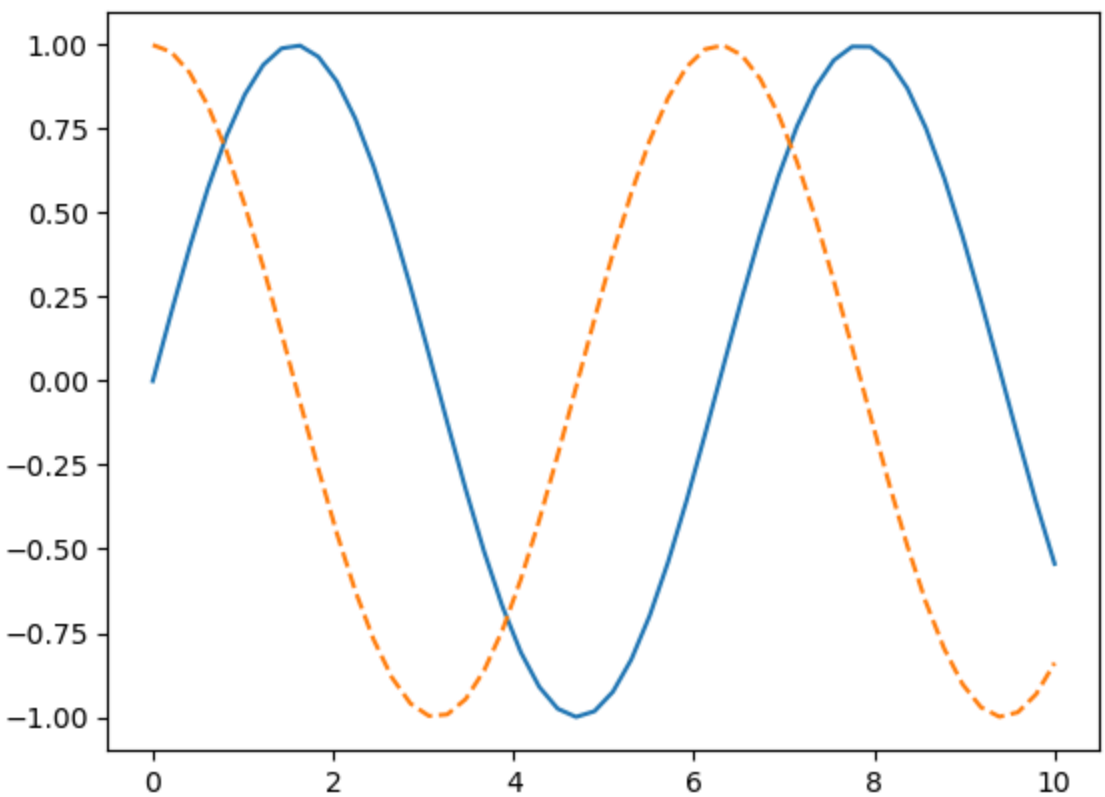
In [3]:
```
import matplotlib.pyplot as plt
```

In [4]:
```
# Import dependencies
import numpy as np
import pandas as pd
```

In [5]:
```
import matplotlib.pyplot as plt
```

In [7]:
```
%matplotlib inline
x1 = np.linspace(0, 10, 50)
```

In [8]:
```
# create a plot figure
#fig = plt.figure()

plt.plot(x1, np.sin(x1), '-')
plt.plot(x1, np.cos(x1), '--')
#plt.plot(x1, np.tan(x1), '--')
plt.show()
```



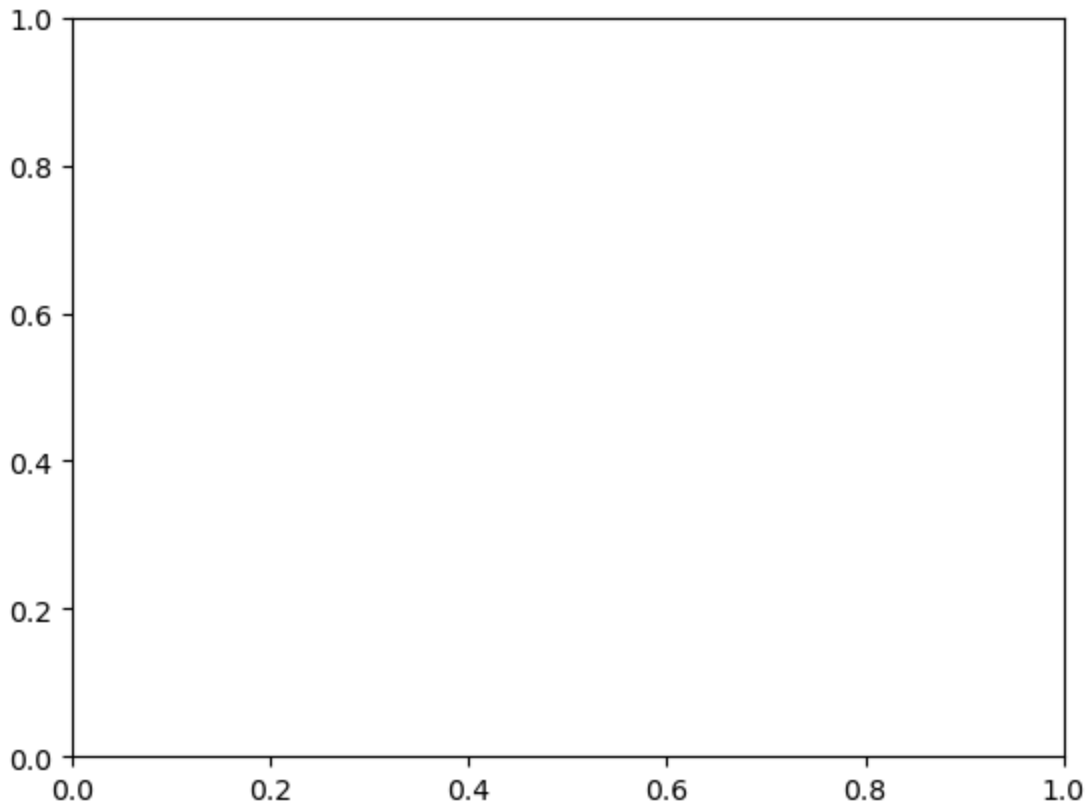
In [9]:
```
plt.gcf ( ) # get current figure
```

Out[9]:
```
<Figure size 640x480 with 0 Axes>
```


```
<Figure size 640x480 with 0 Axes>
```

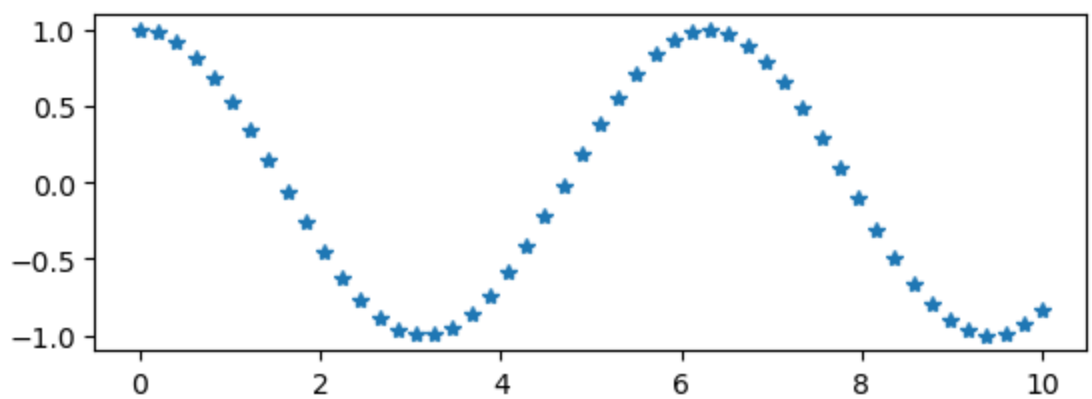
In [10]:
```
plt.gca ( ) # get current axes
```

Out[10]:
```
<Axes: >
```



In [11]:
```
# create the first of two panels and set current axis
plt.subplot(2, 1, 1)   # (rows, columns, panel number)
plt.plot(x1, np.cos(x1), '*')
```

Out[11]:
```
[<matplotlib.lines.Line2D at 0x243e4bd5a90>]
```



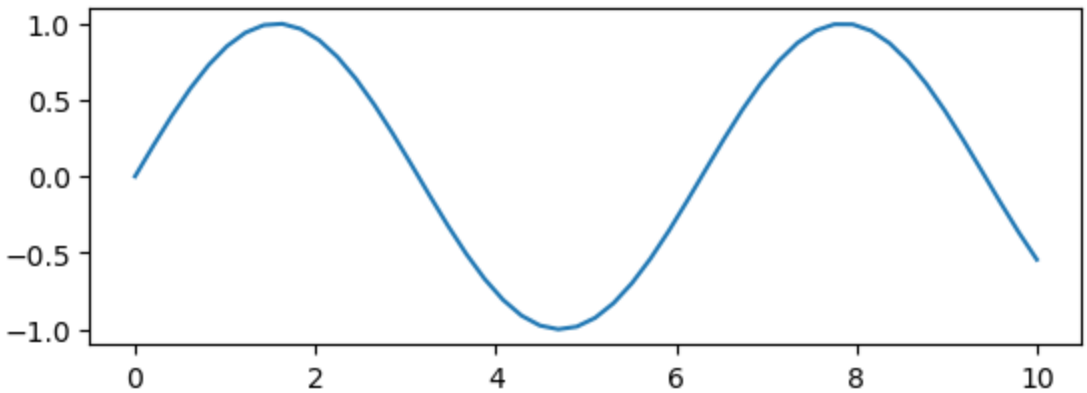
In [12]:
```
# create a plot figure
plt.figure()
```

Out[12]:
```
<Figure size 640x480 with 0 Axes>
```

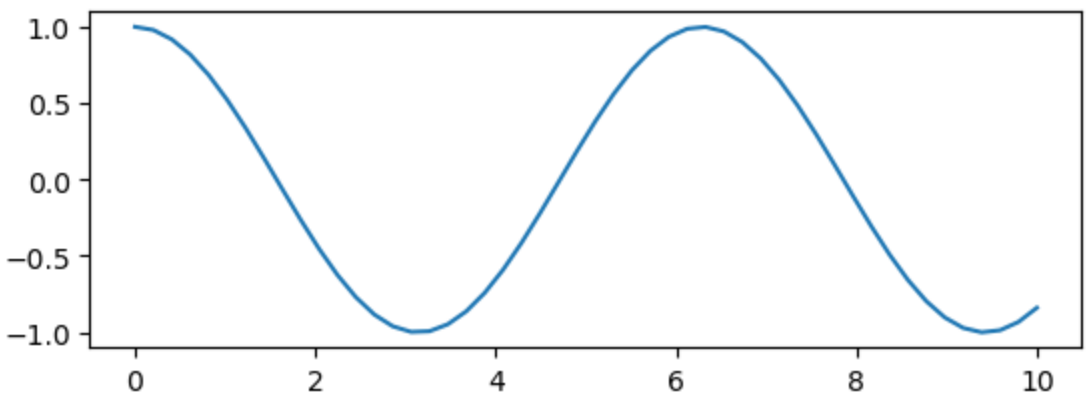
```
<Figure size 640x480 with 0 Axes>
```

In [13]:
```
# create the first of two panels and set current axis
plt.subplot(2, 1, 1)   # (rows, columns, panel number)
plt.plot(x1, np.sin(x1))
```

Out[13]:
```
[<matplotlib.lines.Line2D at 0x243e4da6850>]
```



In [14]:
```
# create the second of two panels and set current axis
plt.subplot(2, 1, 2)   # (rows, columns, panel number)
plt.plot(x1, np.cos(x1));
```

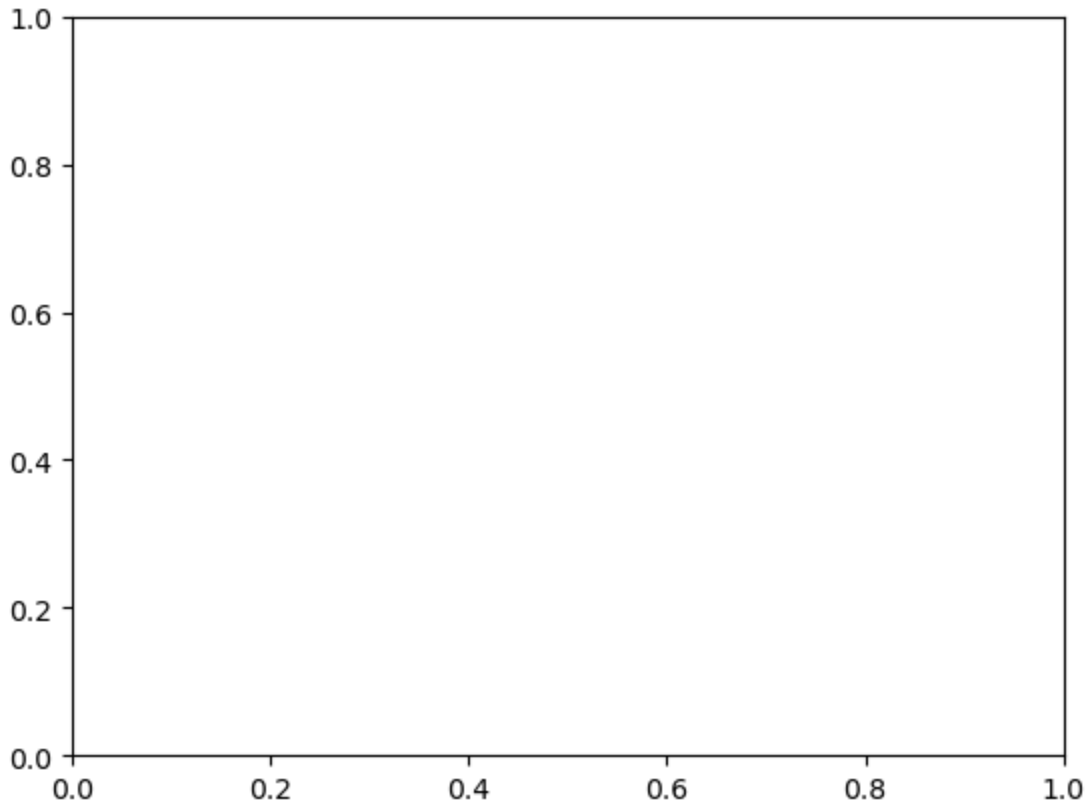


In [15]:
```
# get current figure information

print(plt.gcf())
```

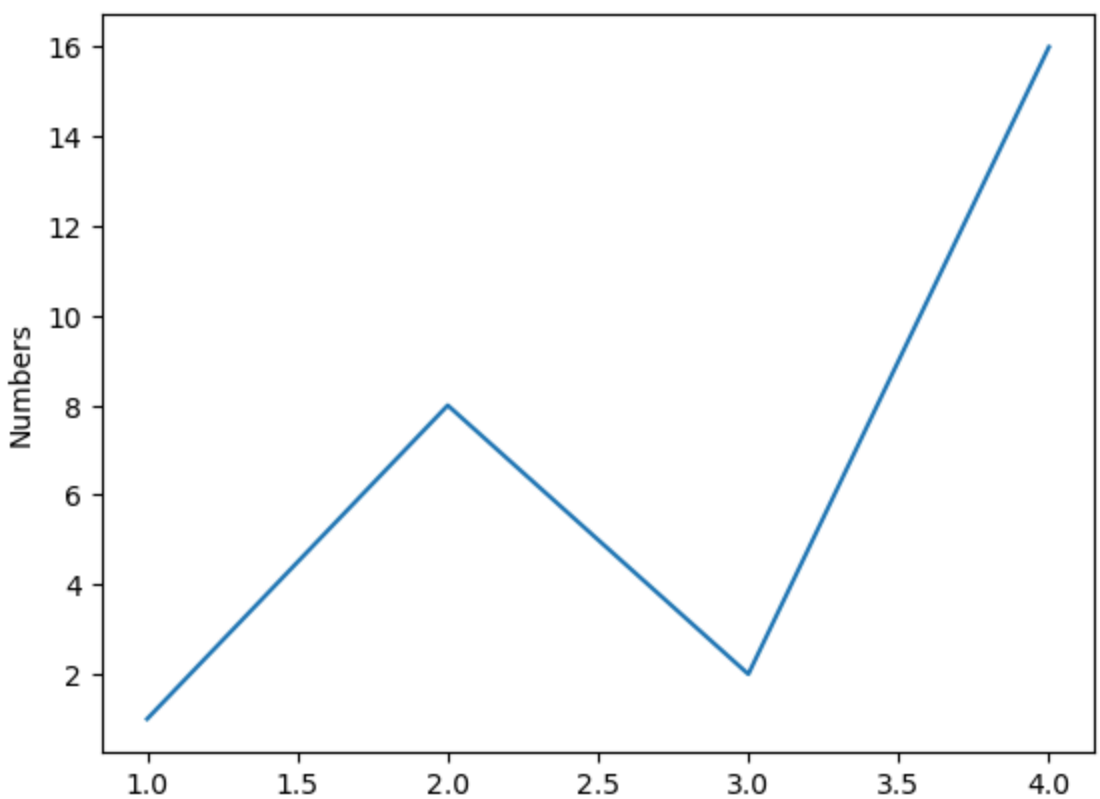
```
Figure(640x480)
<Figure size 640x480 with 0 Axes>
```

In [16]:
```
# get current axis information

print(plt.gca())
```

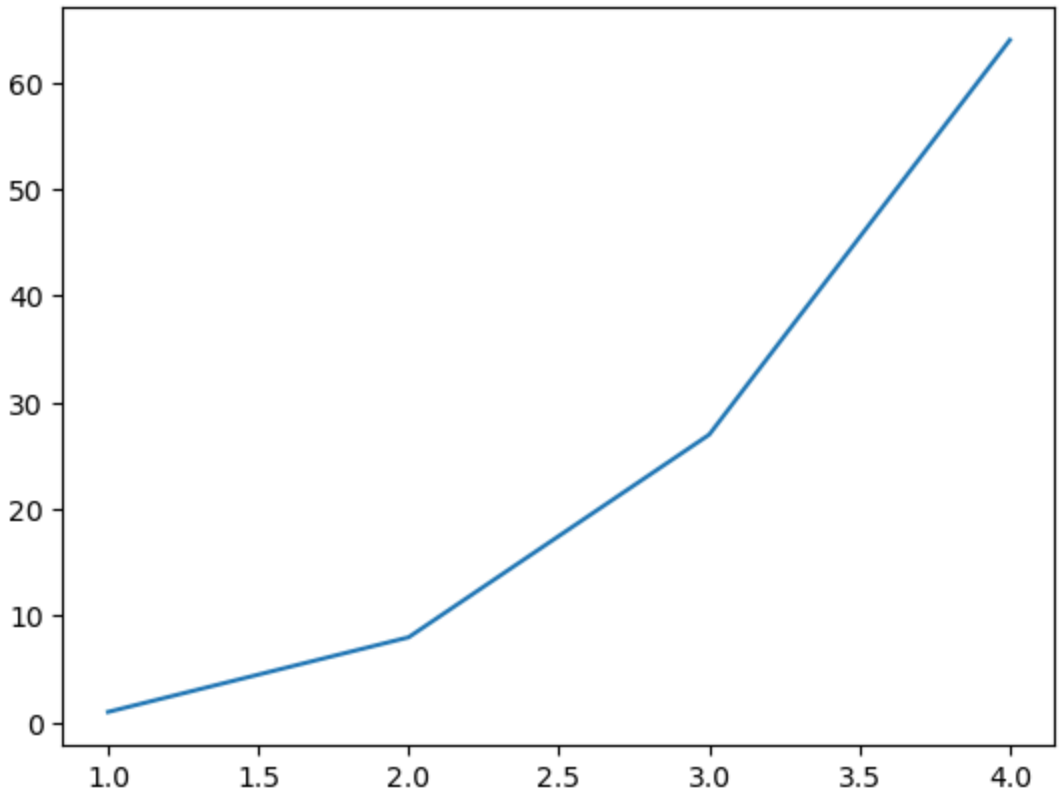
```
Axes(0.125,0.11;0.775x0.77)
```



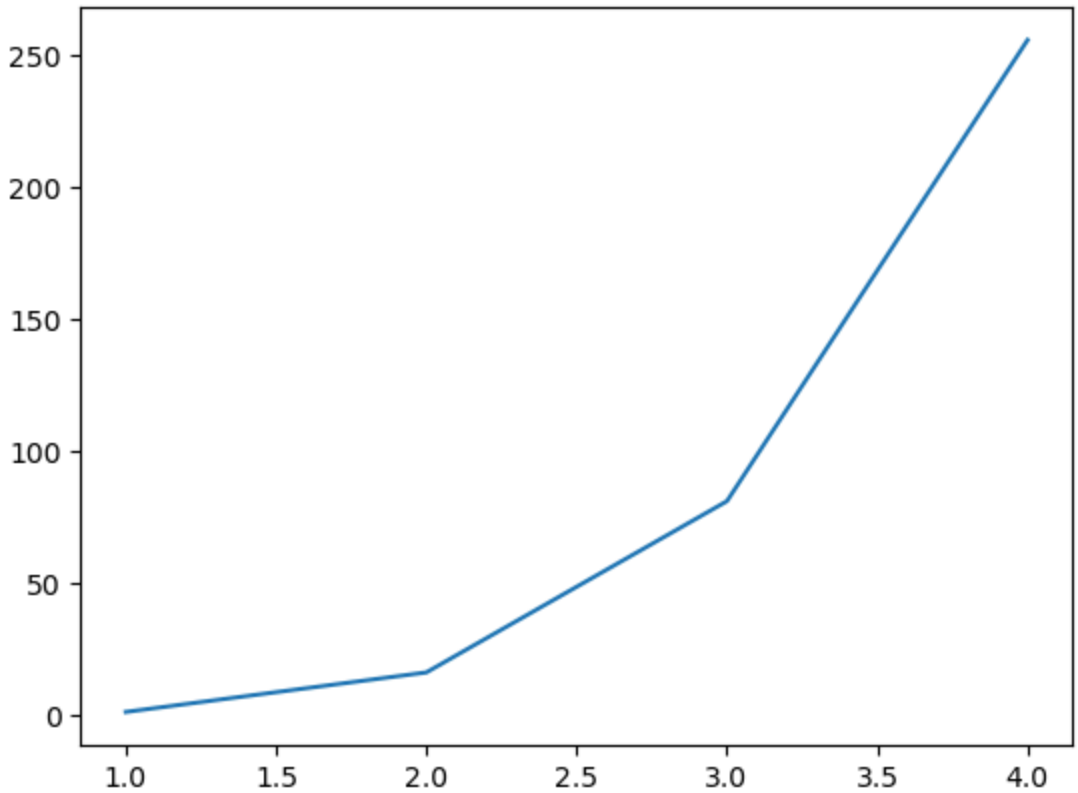
In [17]:
```
plt.plot([1,2,3,4], [1,8,2,16])
plt.ylabel('Numbers')
plt.show()
```



In [18]:
```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4], [1, 8, 27, 64])
plt.show()
```



In [19]:
```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4], [1, 16, 81, 256])
plt.show()
```

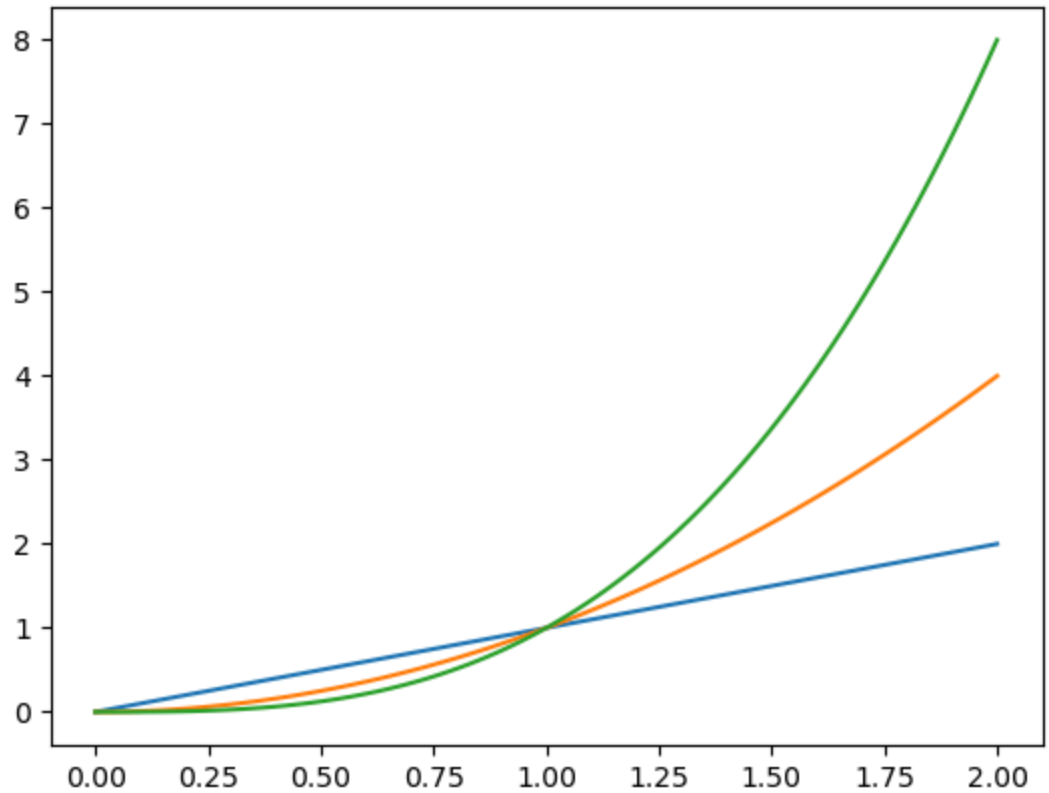


State-machine interface

In [20]:
```
x = np.linspace(0, 2, 100)
```

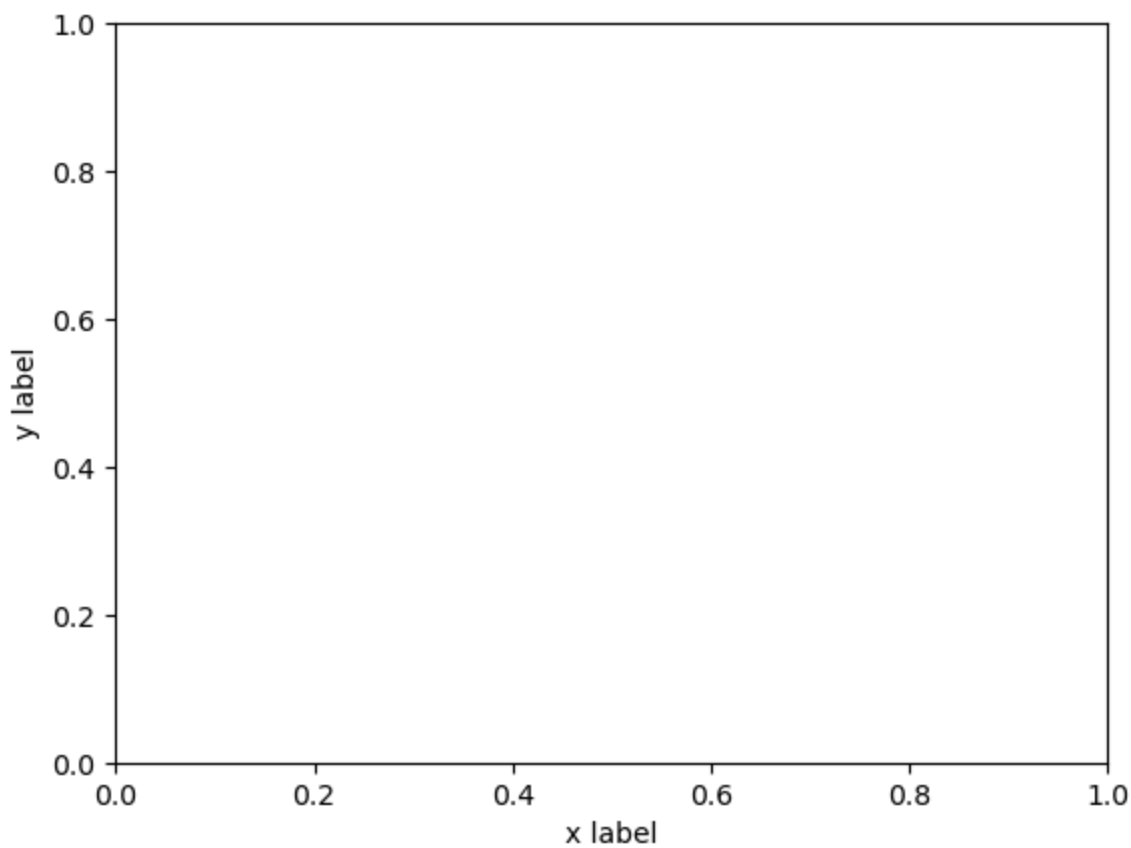
In [21]:
```
plt.plot(x, x, label='linear')
plt.plot(x, x**2, label='quadratic')
plt.plot(x, x**3, label='cubic')
```

Out[21]:
```
[<matplotlib.lines.Line2D at 0x243e5120cd0>]
```



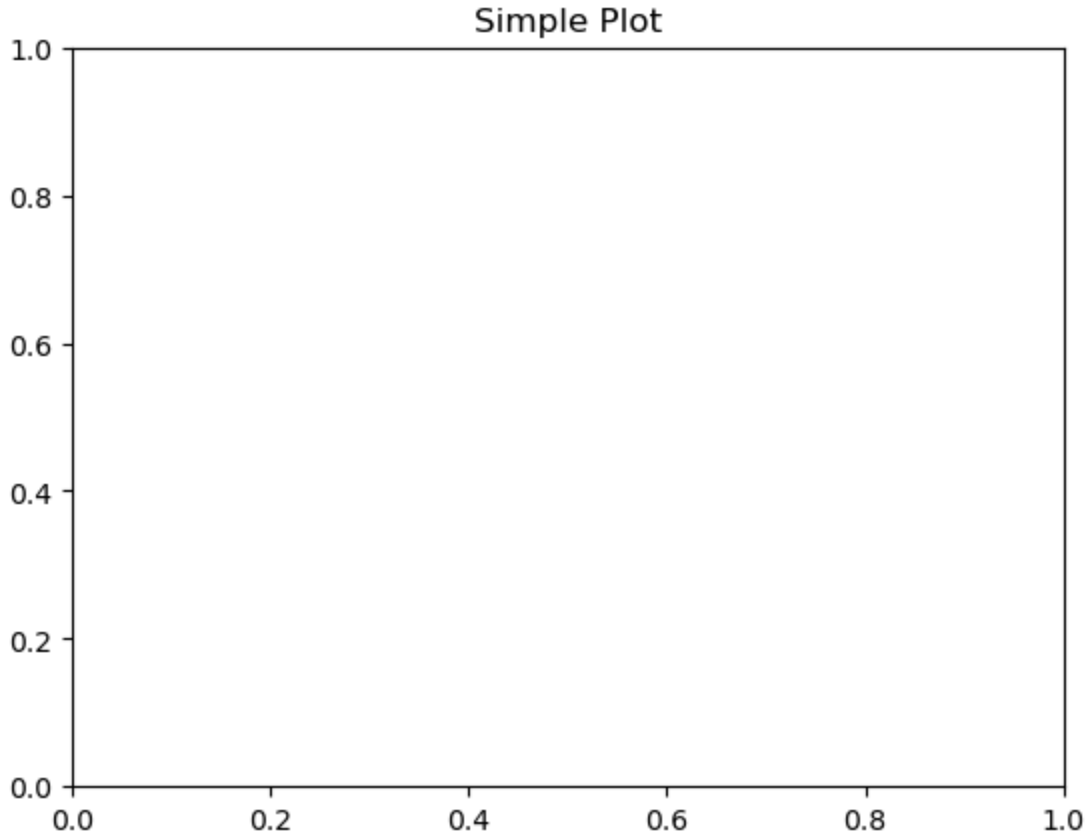
In [22]:
```
plt.xlabel('x label')
plt.ylabel('y label')
```

Out[22]: Text(0, 0.5, 'y label')



In [23]:
```
plt.title("Simple Plot")
```

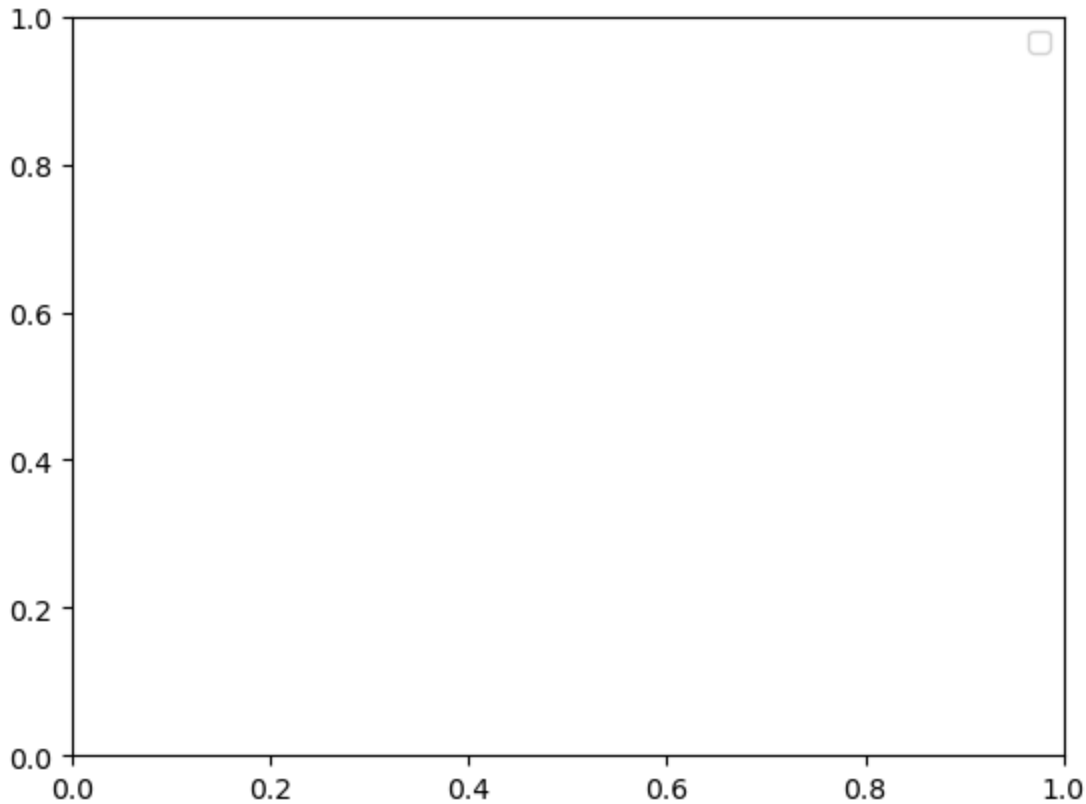
Out[23]:
```
Text(0.5, 1.0, 'Simple Plot')
```



In [24]:
```
plt.legend()
```

```
C:\Users\srini\AppData\Local\Temp\ipykernel_1604\4061938096.py:1: UserWarning: No ar
tists with labels found to put in legend.  Note that artists whose label start with
an underscore are ignored when legend() is called with no argument.
  plt.legend()
```

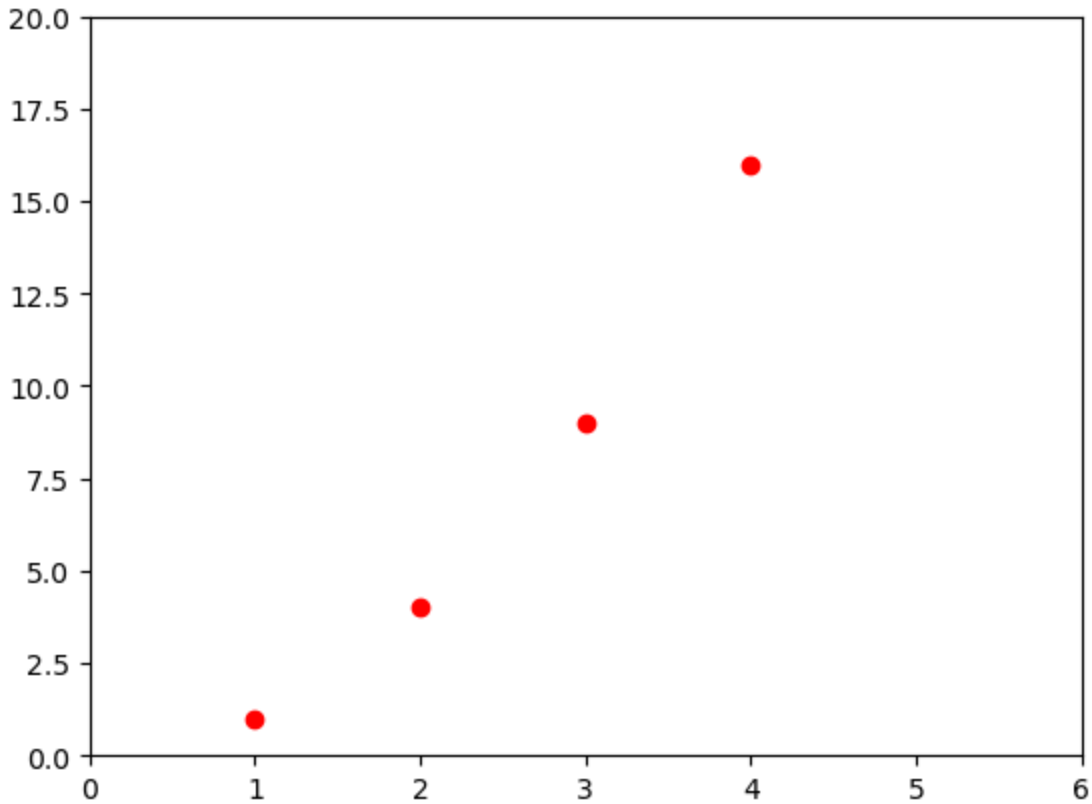
Out[24]:
```
<matplotlib.legend.Legend at 0x243e4914d70>
```



In [26]:
```
plt.show()
```

Formatting the style of plot

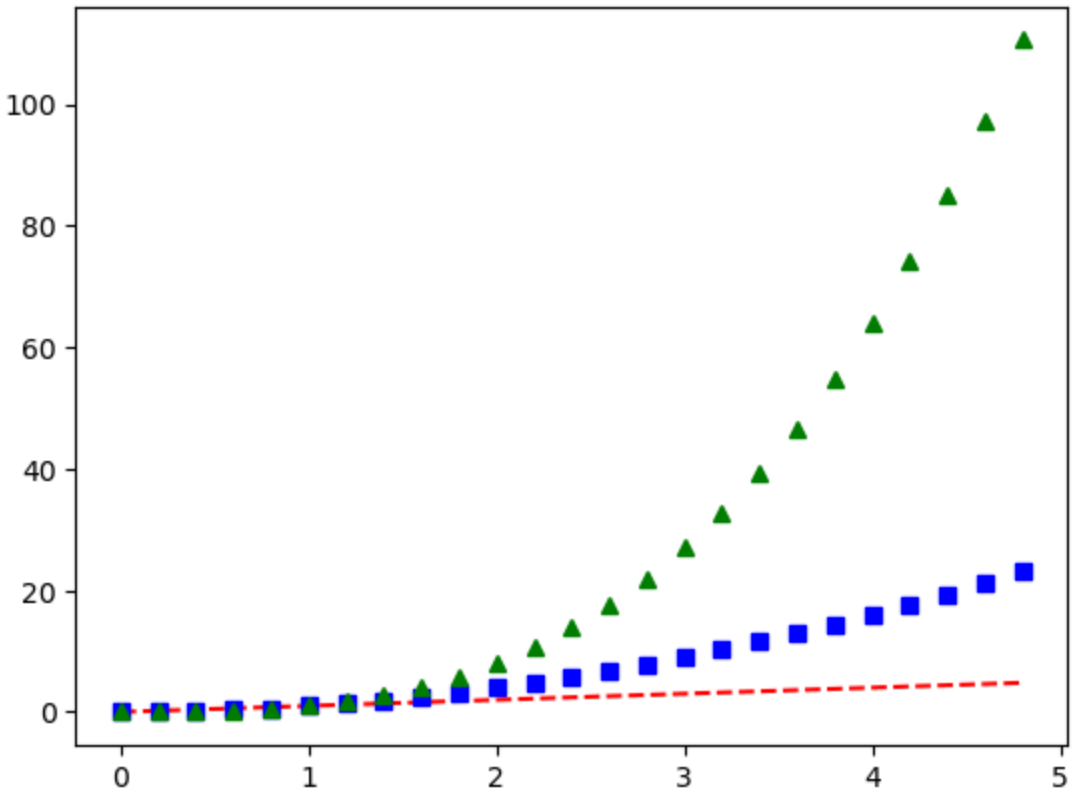
In [31]:
```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
plt.axis([0, 6, 0, 20])
plt.show()
```



Working with Numpy arrays

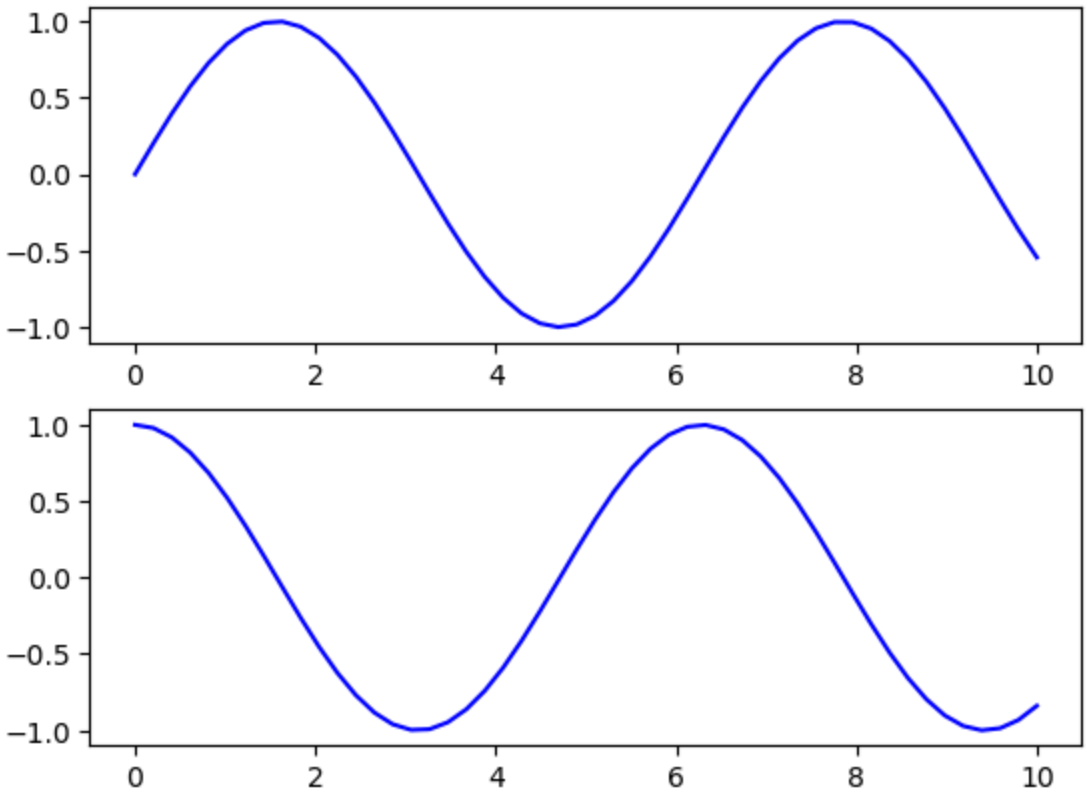
In [32]:
```
# evenly sampled time at 200ms intervals
t = np.arange(0., 5., 0.2)
```

In [33]:
```
# red dashes, blue squares and green triangles
plt.plot(t, t, 'r--', t, t**2, 'bs', t, t**3, 'g^')
plt.show()
```



Object-Oriented API

In [38]:
```
# First create a grid of plots
# ax will be an array of two Axes objects
fig, ax = plt.subplots(2)


# Call plot() method on the appropriate object
ax[0].plot(x1, np.sin(x1), 'b-')
ax[1].plot(x1, np.cos(x1), 'b-');
```



In [39]:
```
fig = plt.figure()

x2 = np.linspace(0, 5, 10)
y2 = x2 ** 2

axes = fig.add_axes([0.1, 0.1, 0.8, 0.8])

axes.plot(x2, y2, 'r')

axes.set_xlabel('x2')
axes.set_ylabel('y2')
axes.set_title('title');
```

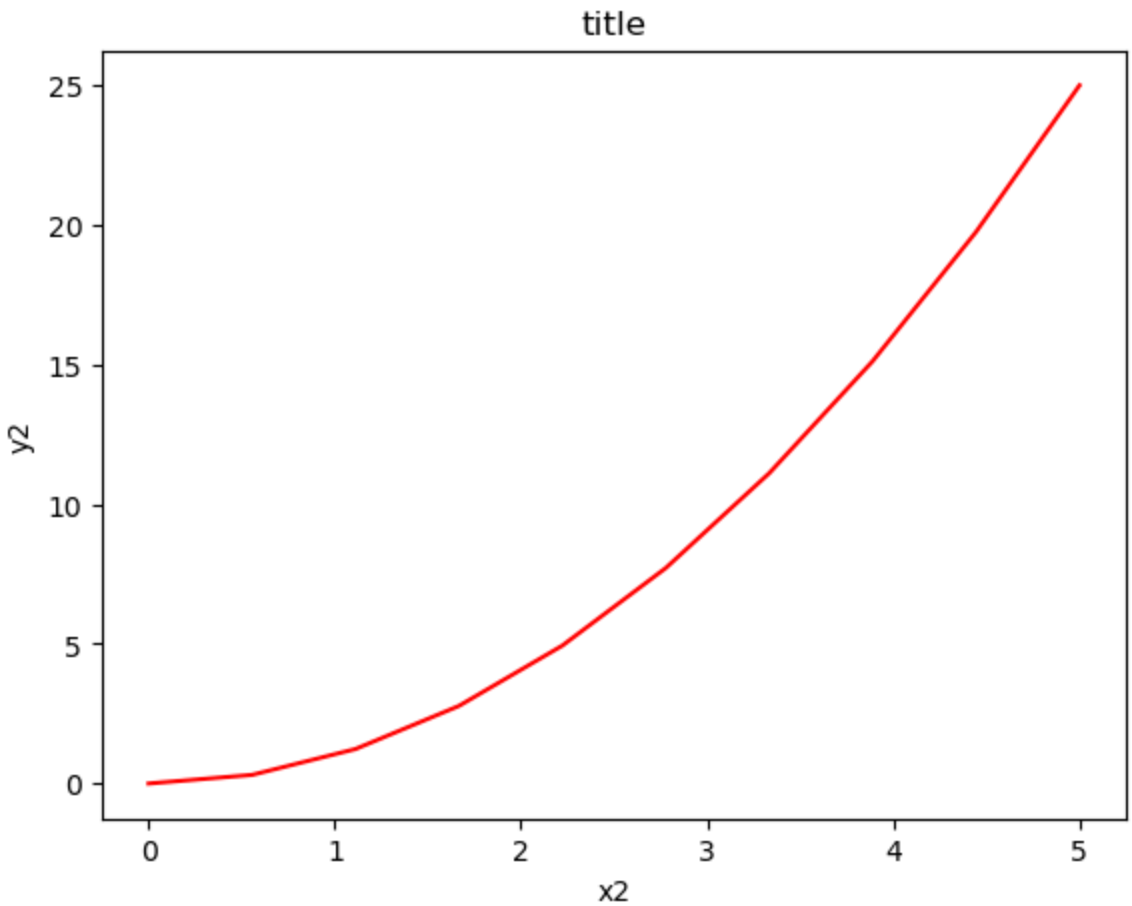


Figure and Axes

In [40]:
```
fig = plt.figure()
```

```
<Figure size 640x480 with 0 Axes>
```

In [41]:
```
ax = plt.axes()
```

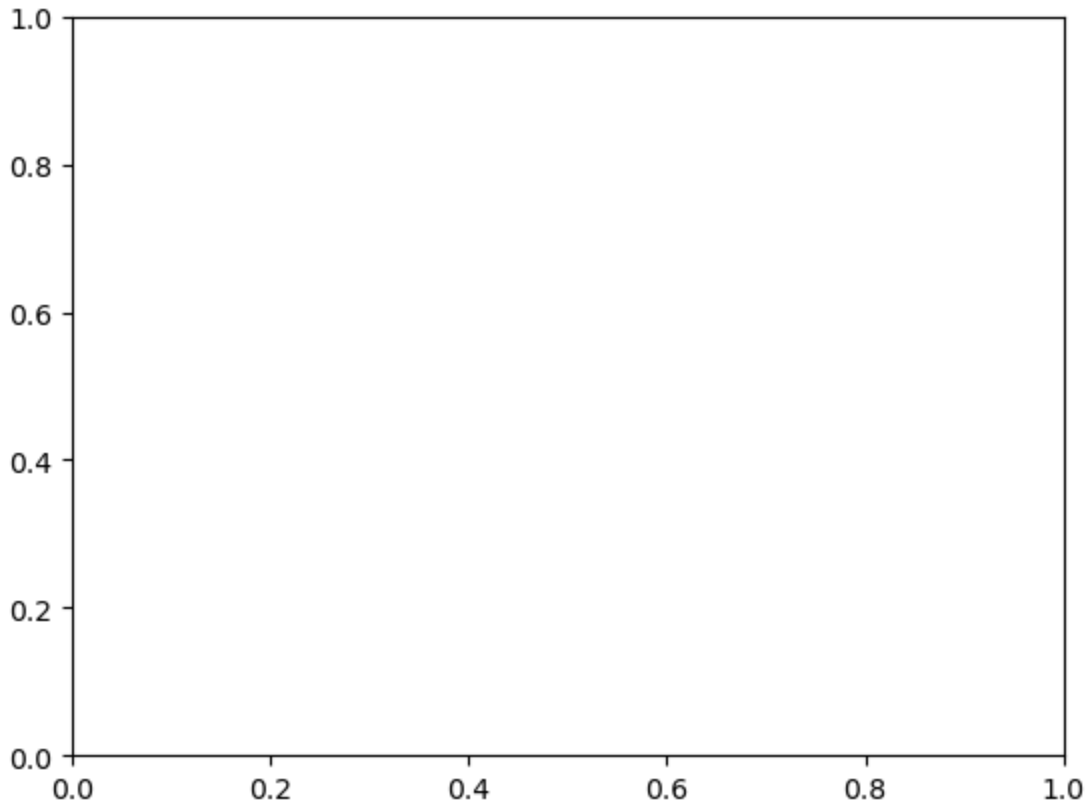
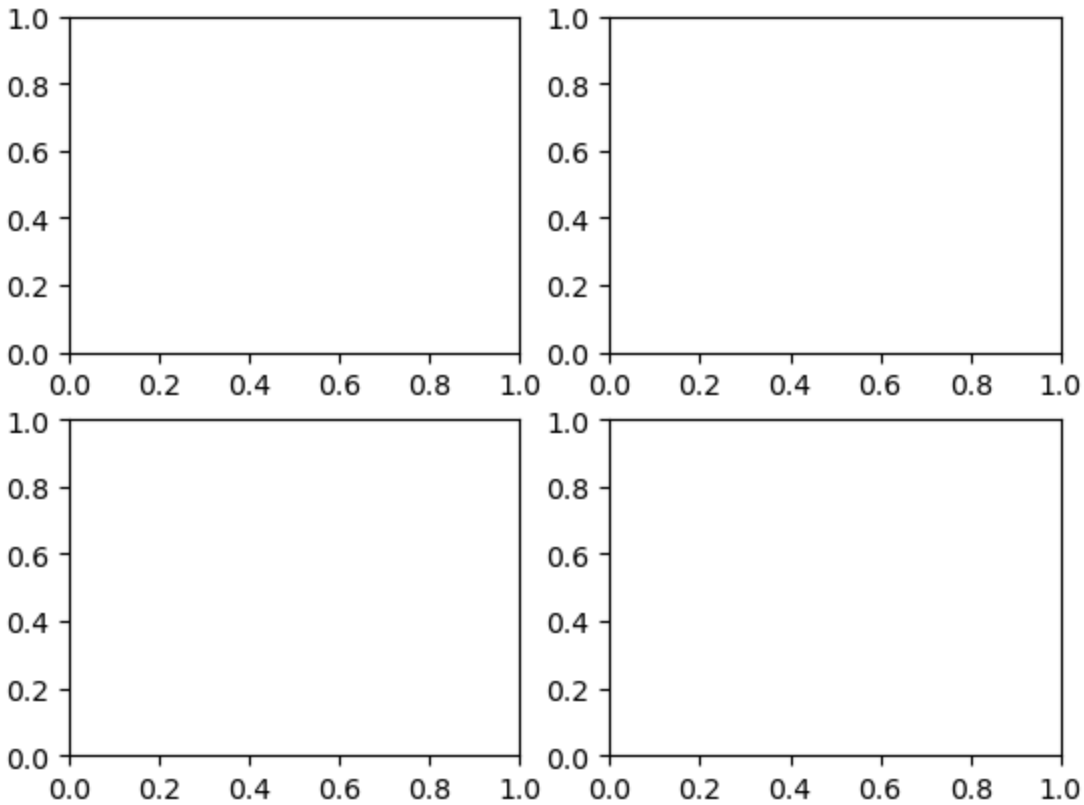


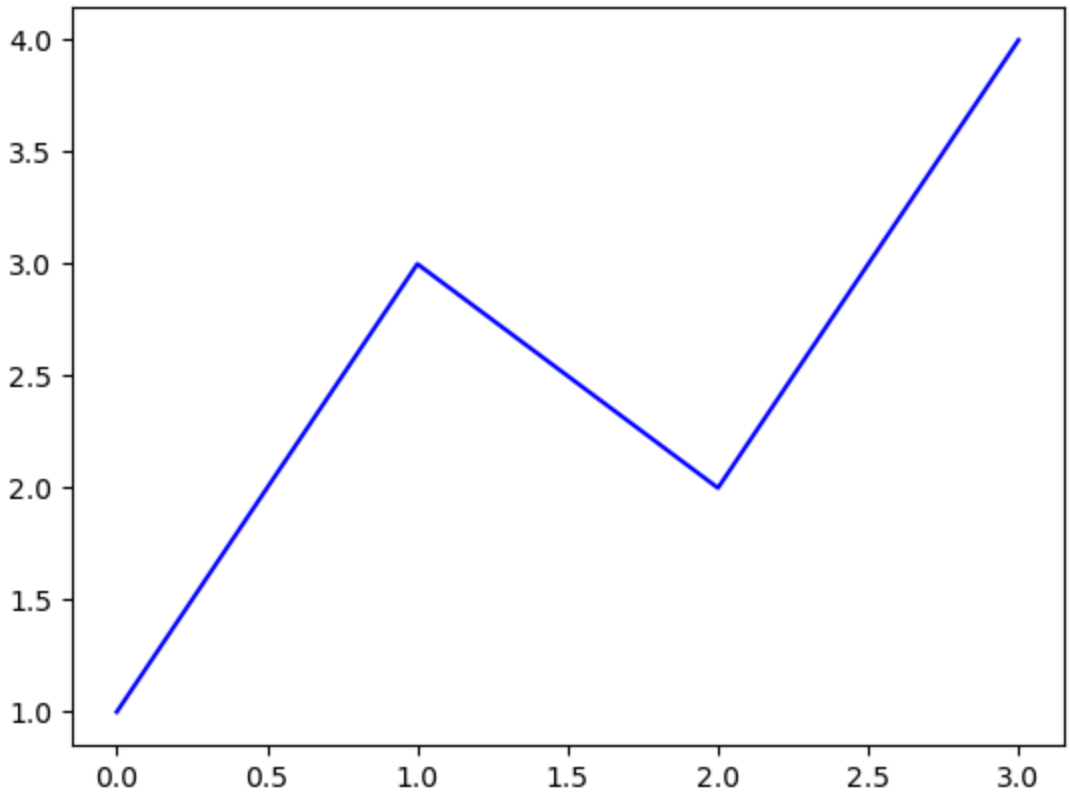
Figure and Subplots

In [50]:
```
fig = plt.figure()
ax1 = fig.add_subplot(2, 2, 1)
ax2 = fig.add_subplot(2, 2, 2)
ax3 = fig.add_subplot(2, 2, 3)
ax4 = fig.add_subplot(2, 2, 4)
```



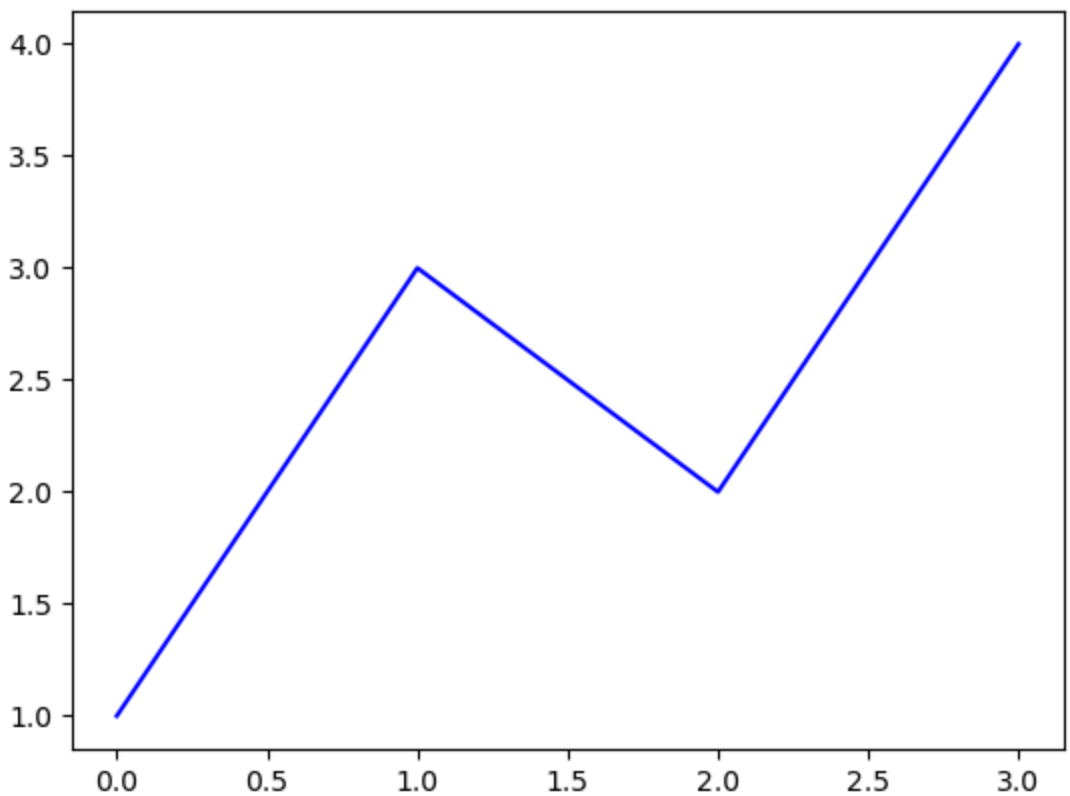
First plot with Matplotlib

In [51]:
```
plt.plot([1, 3, 2, 4], 'b-')

plt.show( )
```



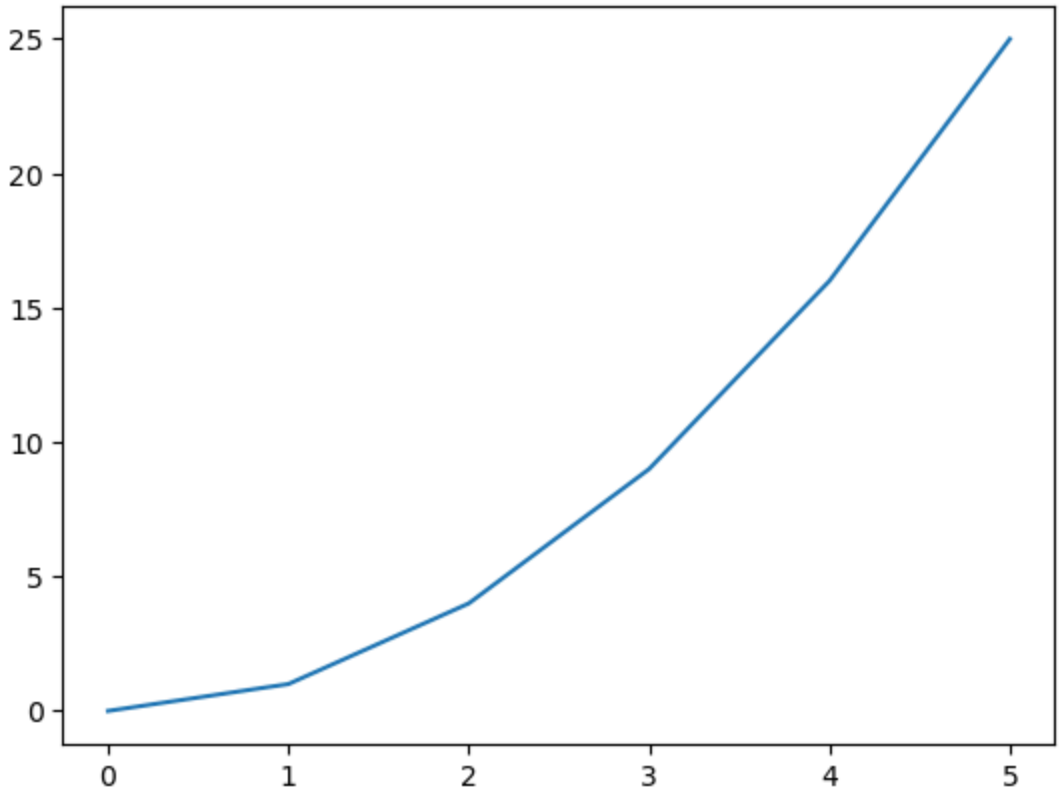
In [52]:
```
plt.plot([1, 3, 2, 4], 'b-')
```

Out[52]:
```
[<matplotlib.lines.Line2D at 0x243e5180cd0>]
```

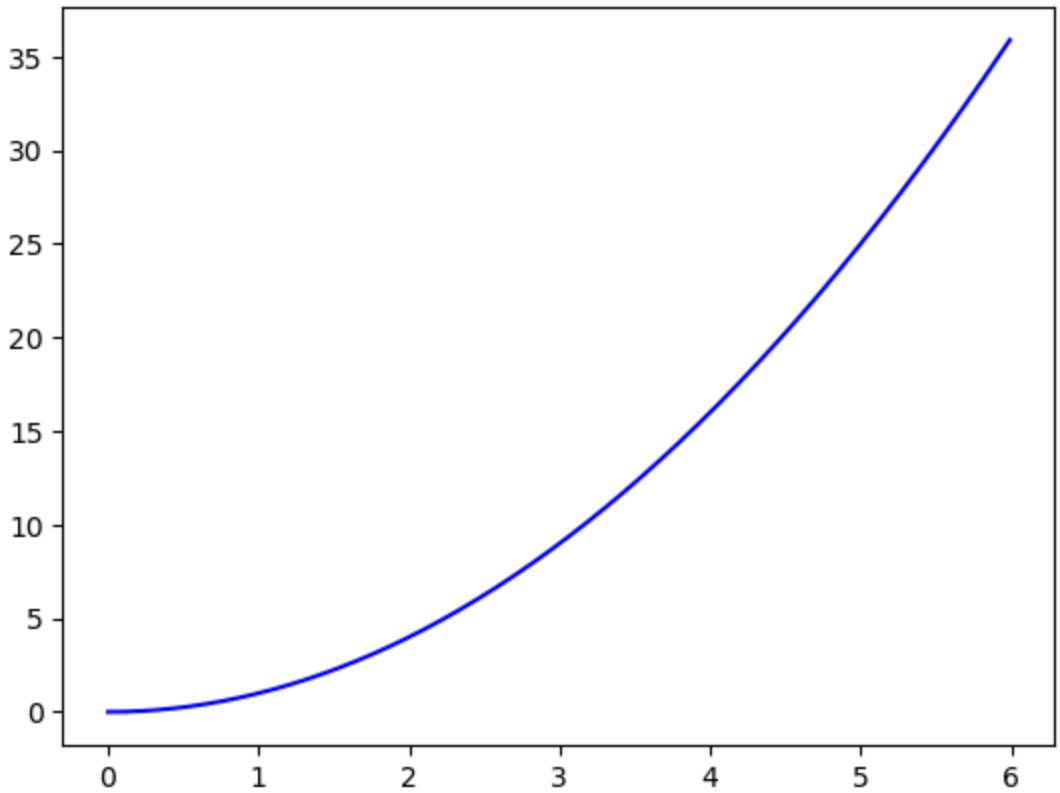


In [53]:
```
x3 = range(6)
```

```
plt.plot(x3, [xi**2 for xi in x3])

plt.show()
```

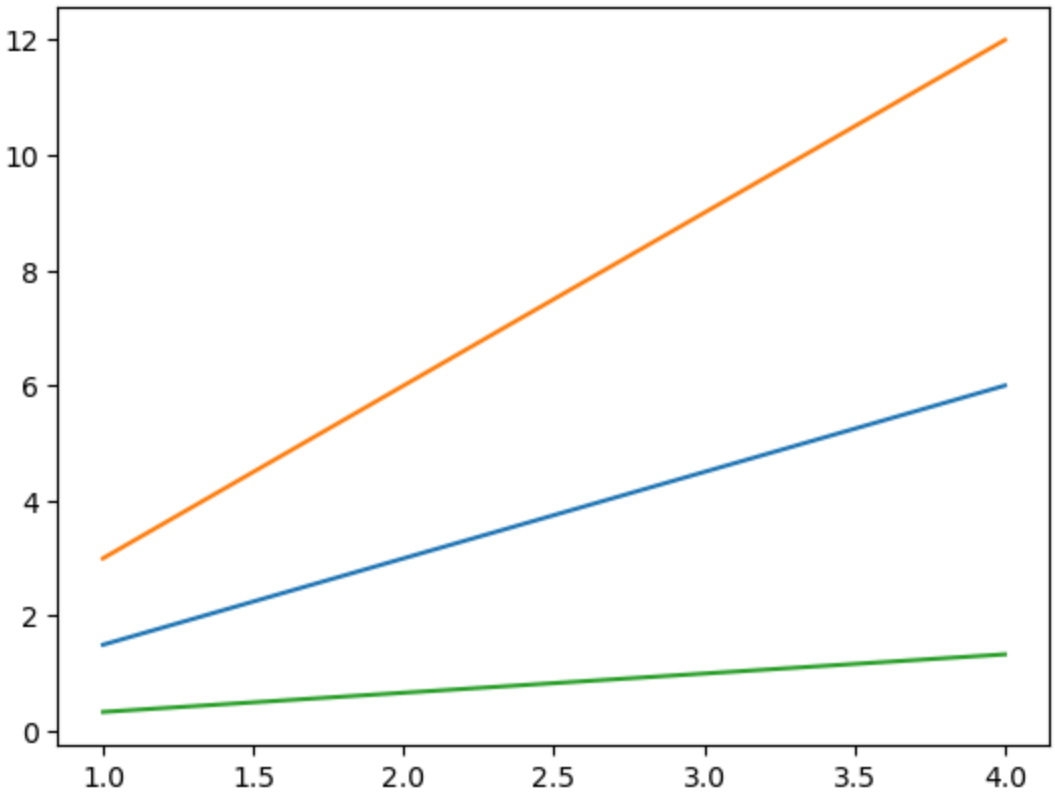


In [54]:
```
x3 = np.arange(0.0, 6.0, 0.01)

plt.plot(x3, [xi**2 for xi in x3], 'b-')

plt.show()
```



Multiline Plots

In [55]:
```
x4 = range(1, 5)
plt.plot(x4, [xi*1.5 for xi in x4])
plt.plot(x4, [xi*3 for xi in x4])
plt.plot(x4, [xi/3.0 for xi in x4])
plt.show()
```

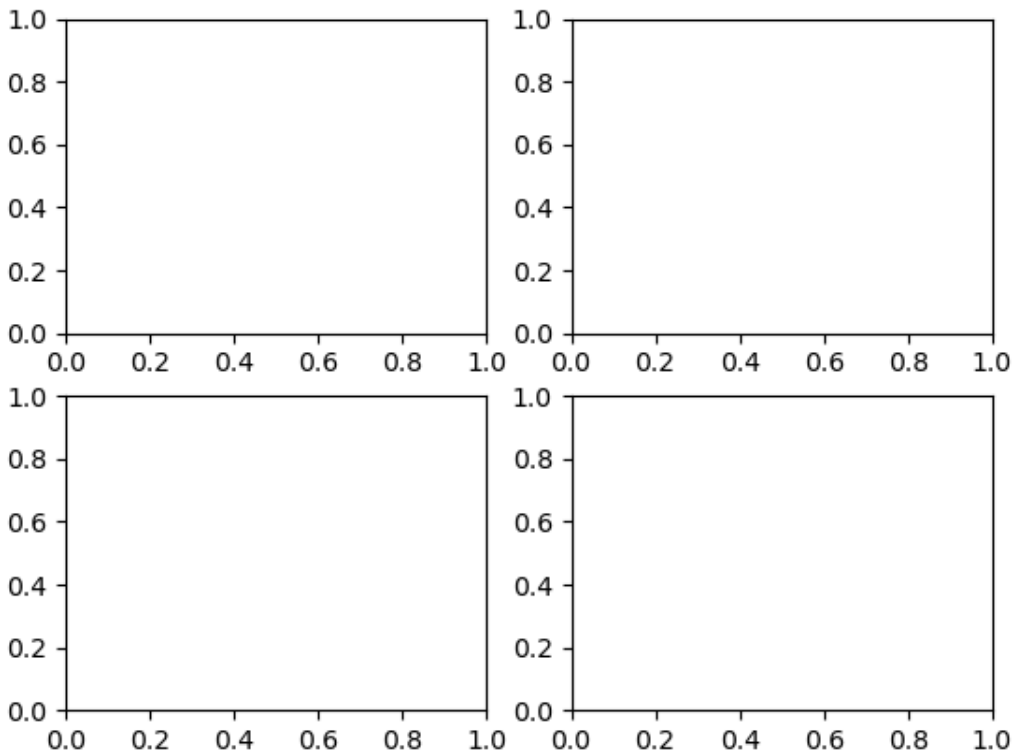


Saving the Plot

In [59]:
```
# Saving the figure

fig.savefig('plot1.png')
```

In [60]:
```
# Explore the contents of figure

from IPython.display import Image

Image('plot1.png')
```

Out[60]:



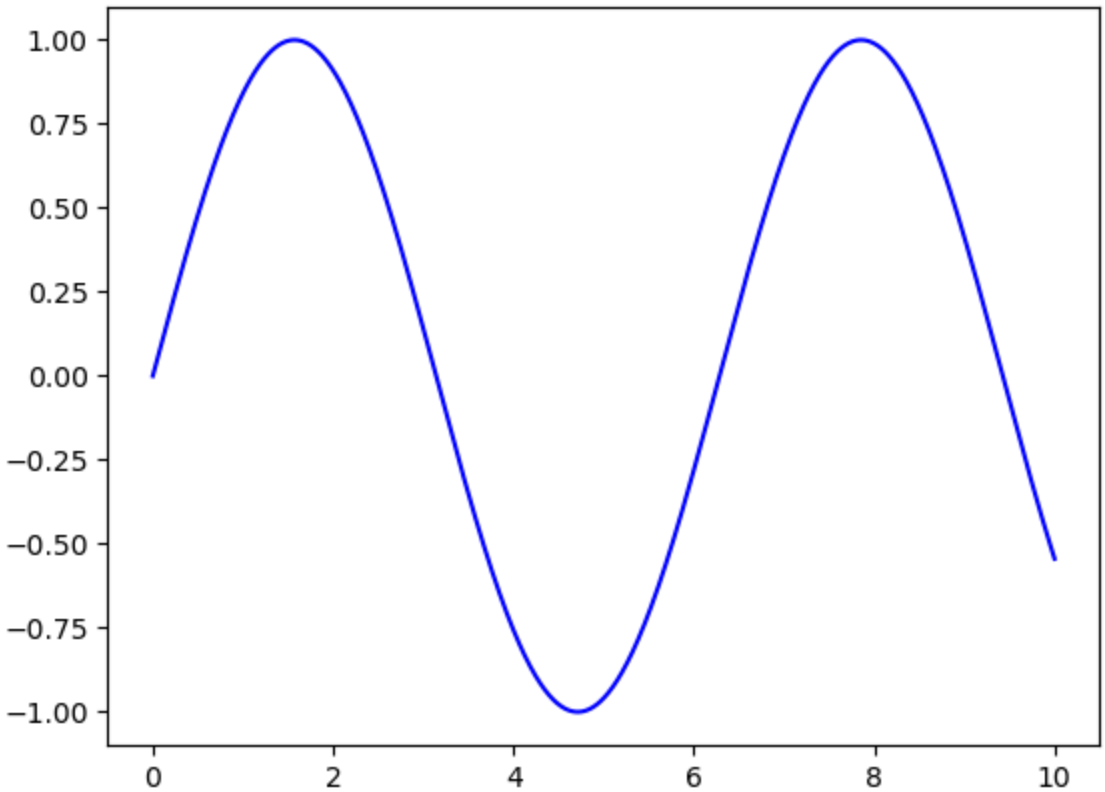
In [61]:
```
# Explore supported file formats


fig.canvas.get_supported_filetypes()
```

Out[61]:
```
{'eps': 'Encapsulated Postscript',
 'jpg': 'Joint Photographic Experts Group',
 'jpeg': 'Joint Photographic Experts Group',
 'pdf': 'Portable Document Format',
 'pgf': 'PGF code for LaTeX',
 'png': 'Portable Network Graphics',
 'ps': 'Postscript',
 'raw': 'Raw RGBA bitmap',
 'rgba': 'Raw RGBA bitmap',
 'svg': 'Scalable Vector Graphics',
 'svgz': 'Scalable Vector Graphics',
 'tif': 'Tagged Image File Format',
 'tiff': 'Tagged Image File Format',
 'webp': 'WebP Image Format'}
```

line Plot

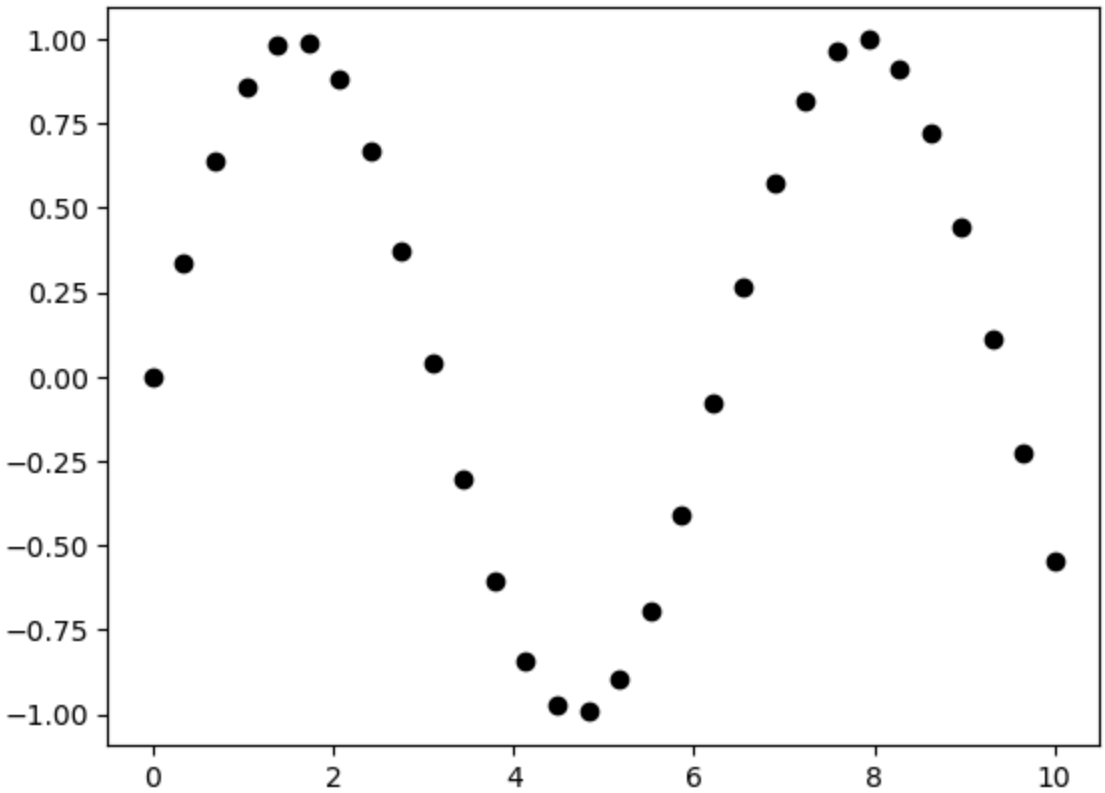
In [62]:
```
# Create figure and axes first
fig = plt.figure()

ax = plt.axes()
```

```
# Declare a variable x5
x5 = np.linspace(0, 10, 1000)


# Plot the sinusoid function
ax.plot(x5, np.sin(x5), 'b-');
```



Scatter Plot

In [68]:
```
x7 = np.linspace(0, 10, 30)

y7 = np.sin(x7)

plt.plot(x7, y7, 'o', color = 'black');
```



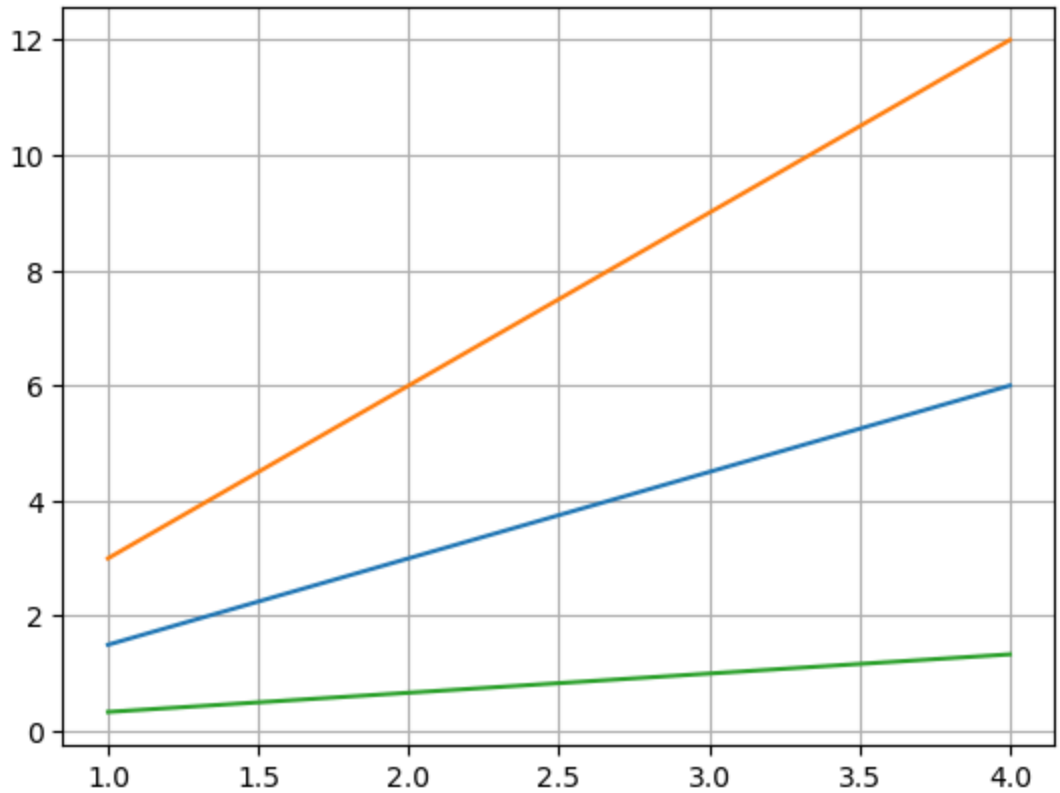
Styles with Matplotlib Plots

In [69]:
```
# View list of all available styles

print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'petroff10', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

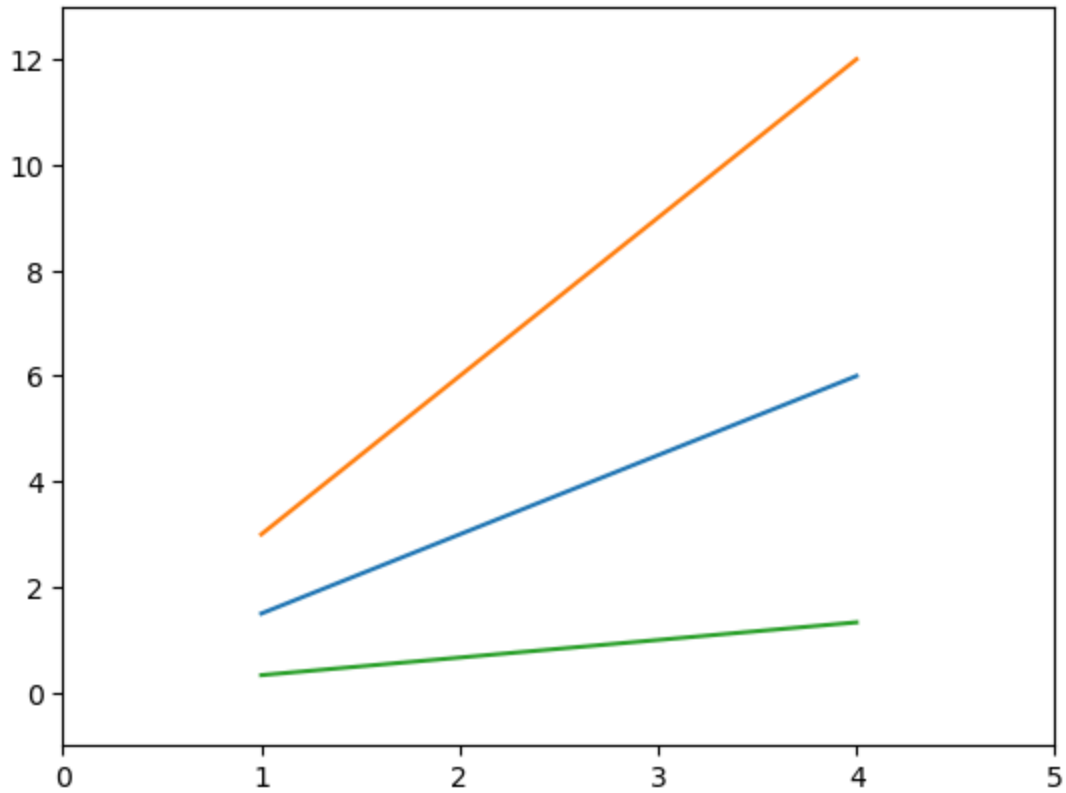
Adding a grid

In [71]:
```
x15 = np.arange(1, 5)

plt.plot(x15, x15*1.5, x15, x15*3.0, x15, x15/3.0)

plt.grid(True)

plt.show()
```



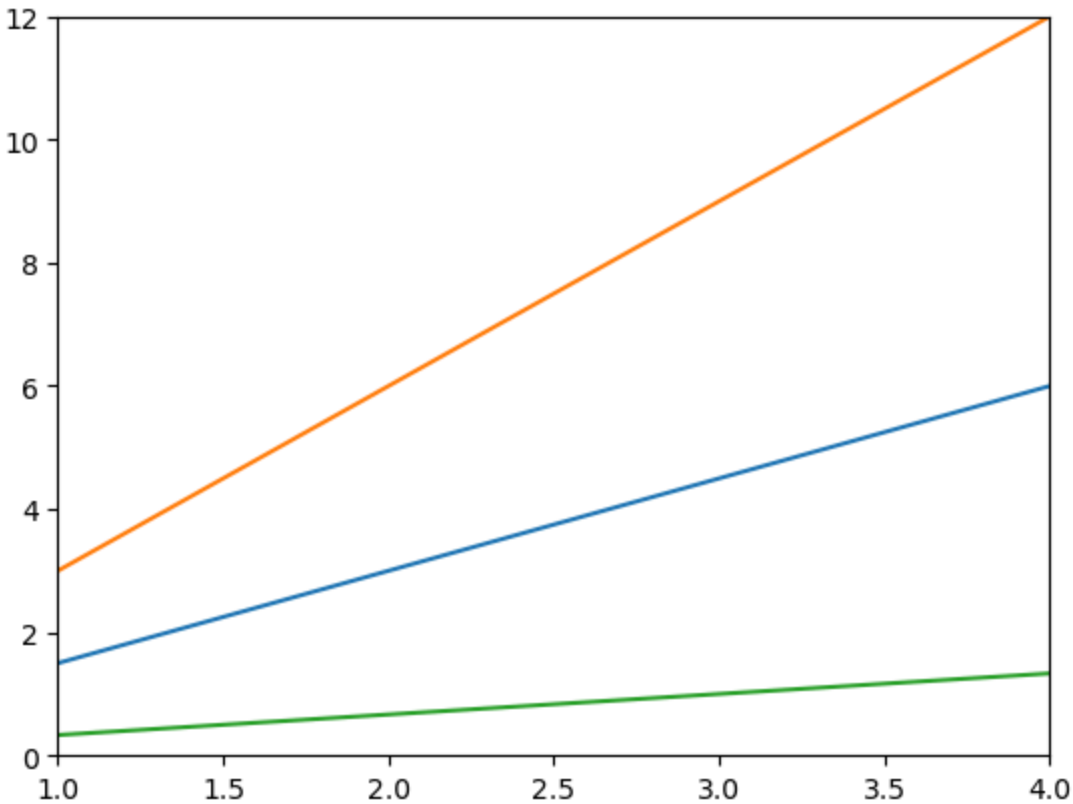
Handling axes

In [72]:
```
x15 = np.arange(1, 5)

plt.plot(x15, x15*1.5, x15, x15*3.0, x15, x15/3.0)

plt.axis()   # shows the current axis limits values

plt.axis([0, 5, -1, 13])

plt.show()
```



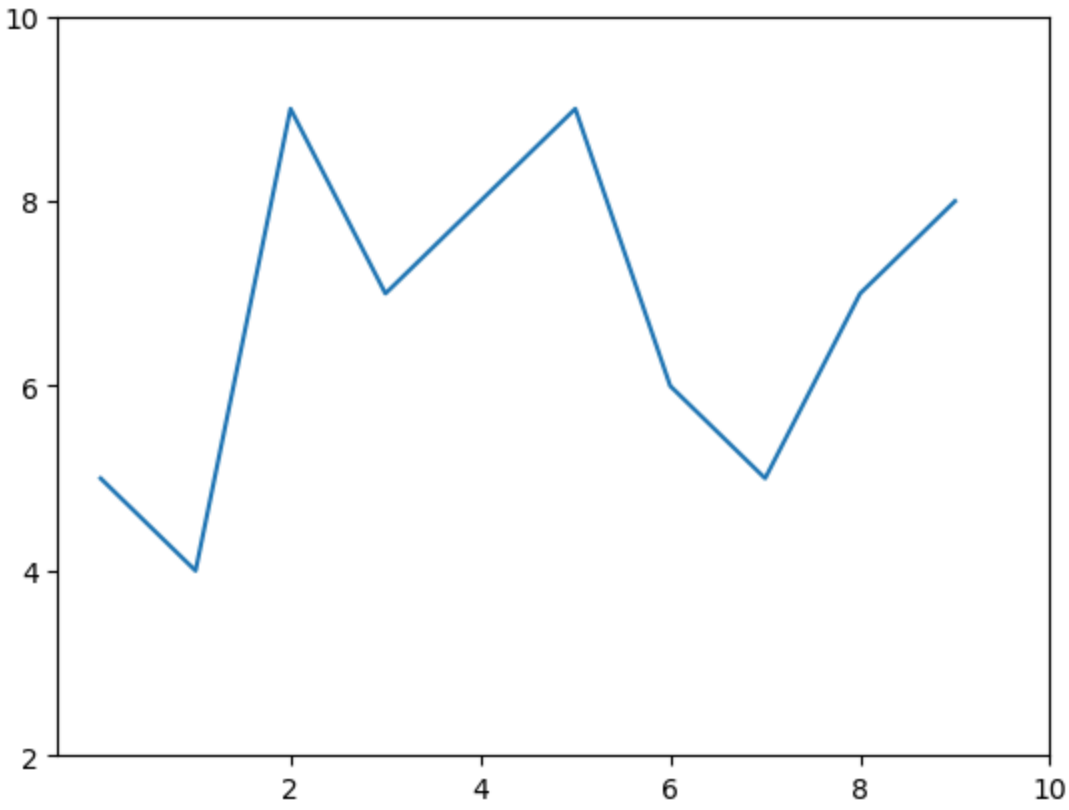
In [73]:
```
x15 = np.arange(1, 5)

plt.plot(x15, x15*1.5, x15, x15*3.0, x15, x15/3.0)

plt.xlim([1.0, 4.0])

plt.ylim([0.0, 12.0])
```

Out[73]: (0.0, 12.0)



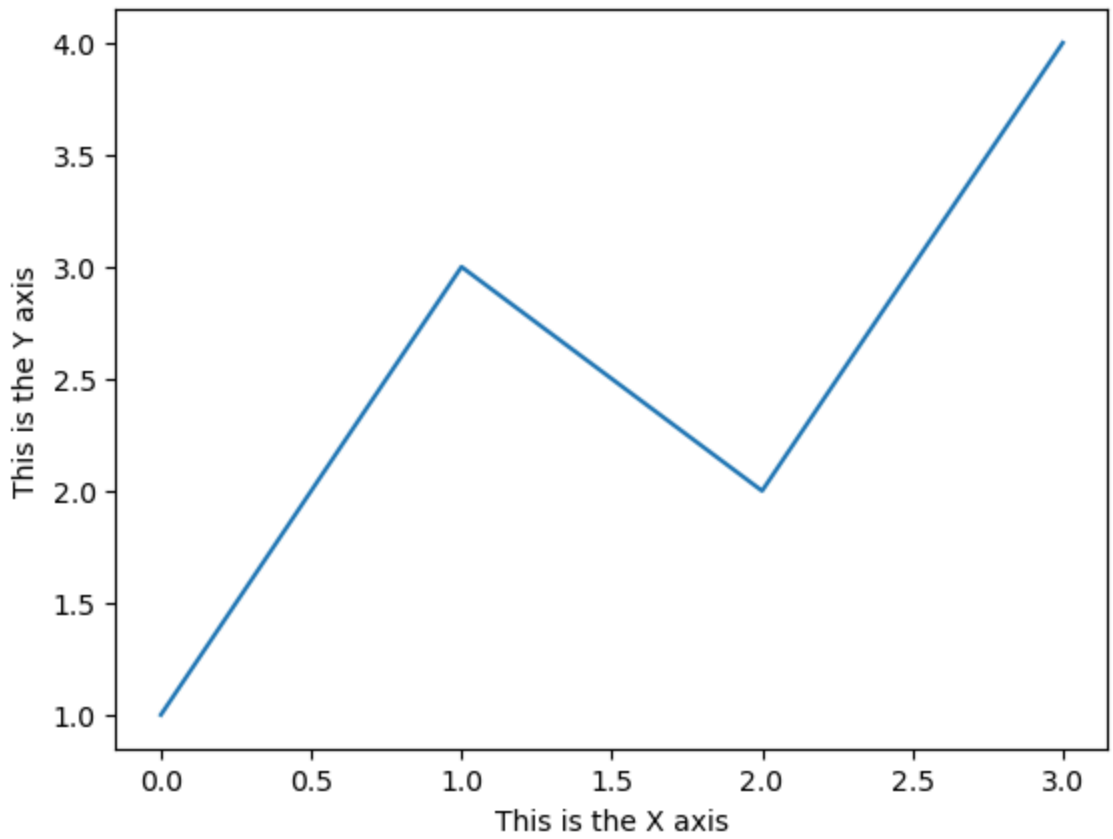
Handling X and Y ticks

In [74]:
```
u = [5, 4, 9, 7, 8, 9, 6, 5, 7, 8]

plt.plot(u)

plt.xticks([2, 4, 6, 8, 10])
plt.yticks([2, 4, 6, 8, 10])

plt.show()
```



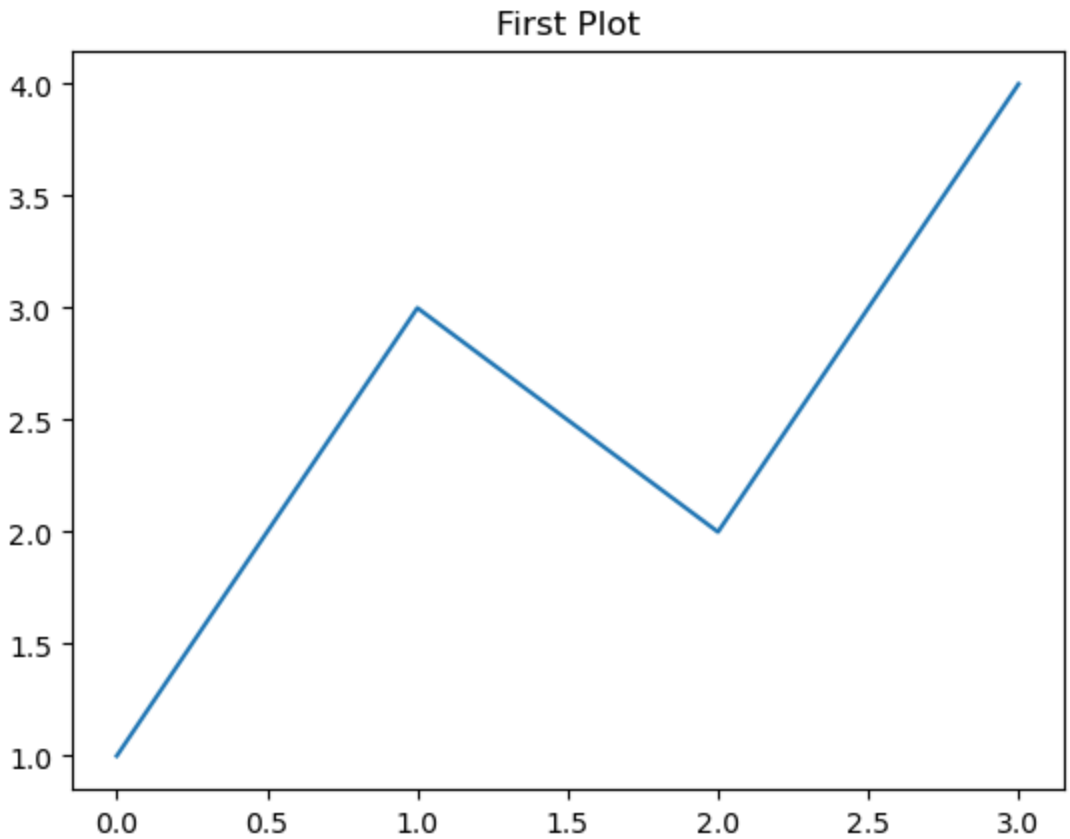
Adding labels

In [75]:
```
plt.plot([1, 3, 2, 4])

plt.xlabel('This is the X axis')

plt.ylabel('This is the Y axis')

plt.show()
```



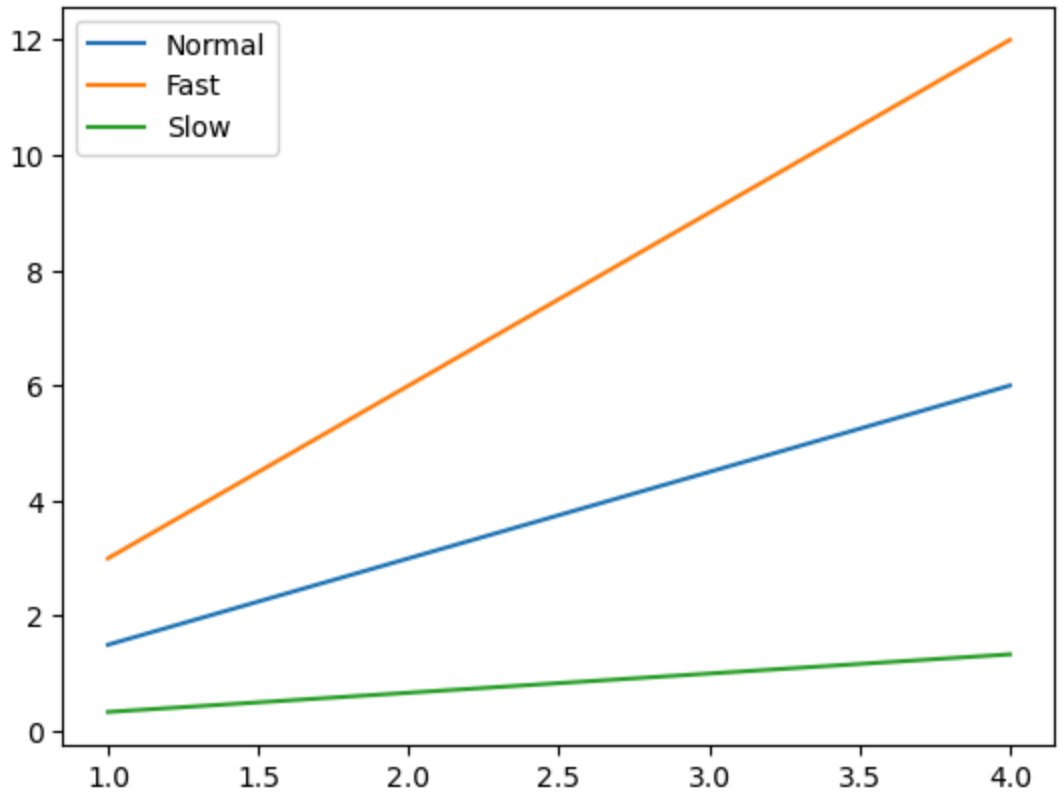
Adding a title

In [76]:
```
plt.plot([1, 3, 2, 4])

plt.title('First Plot')

plt.show()
```

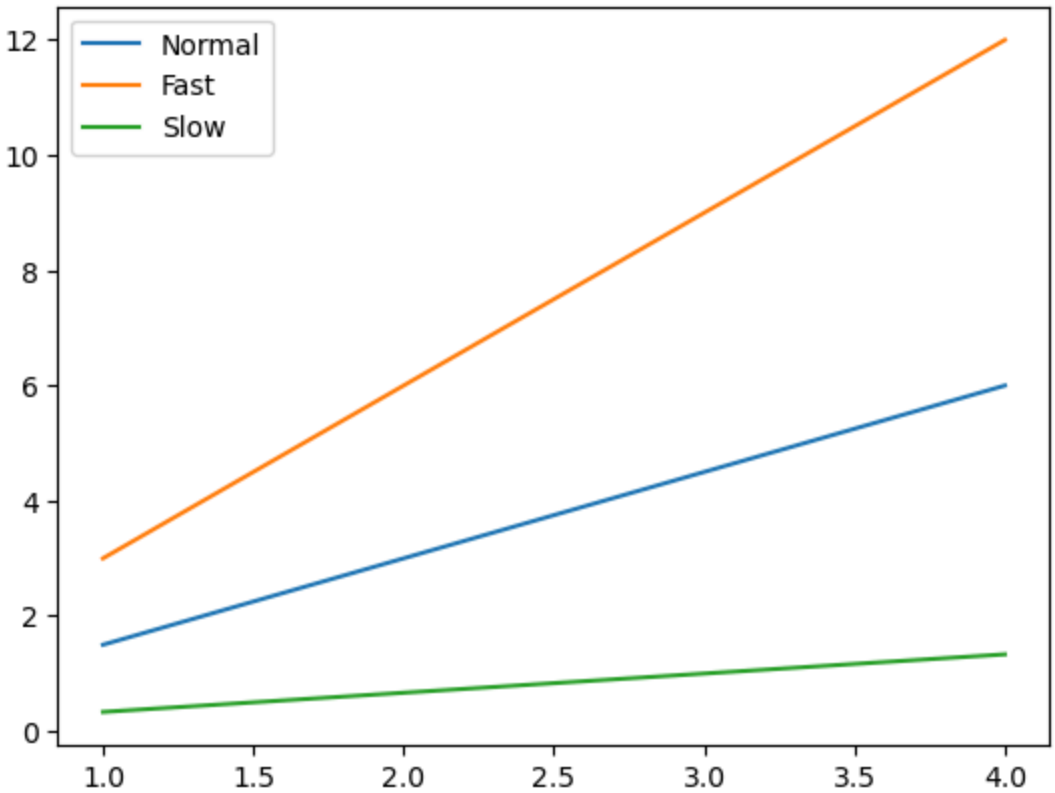


Adding a legend

In [78]:
```
x15 = np.arange(1, 5)

fig, ax = plt.subplots()

ax.plot(x15, x15*1.5)
ax.plot(x15, x15*3.0)
ax.plot(x15, x15/3.0)

ax.legend(['Normal','Fast','Slow']);
```

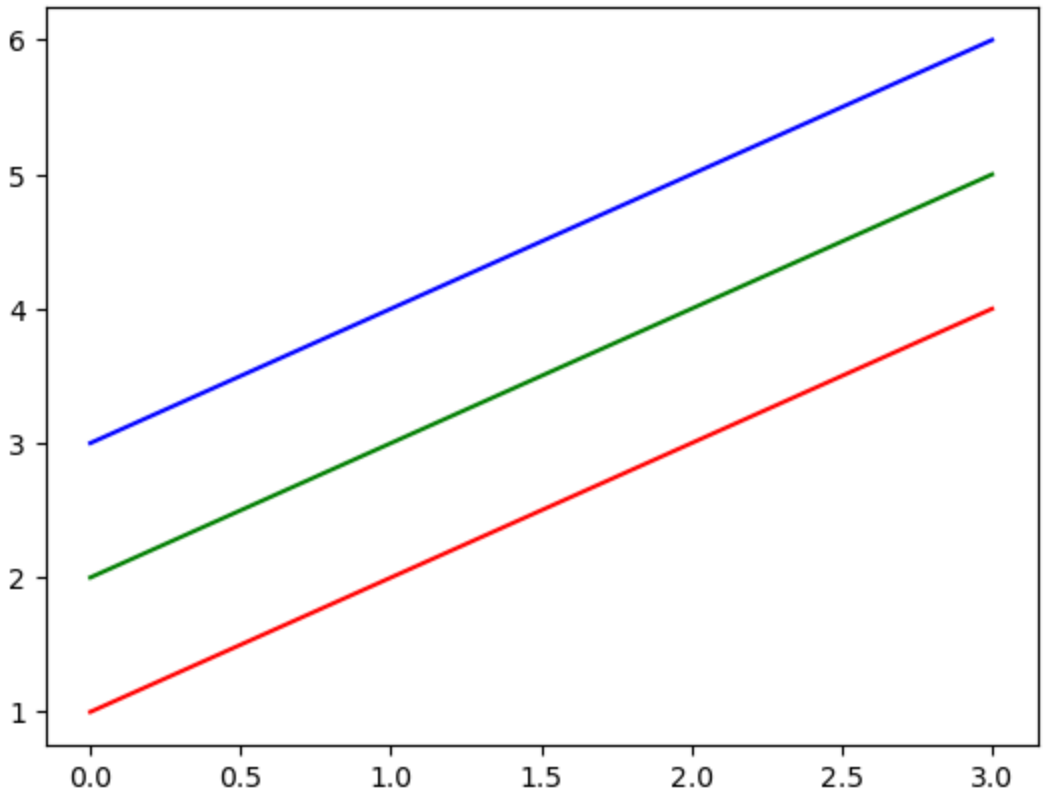


In [79]:
```
x15 = np.arange(1, 5)

fig, ax = plt.subplots()

ax.plot(x15, x15*1.5, label='Normal')
ax.plot(x15, x15*3.0, label='Fast')
ax.plot(x15, x15/3.0, label='Slow')

ax.legend();
```



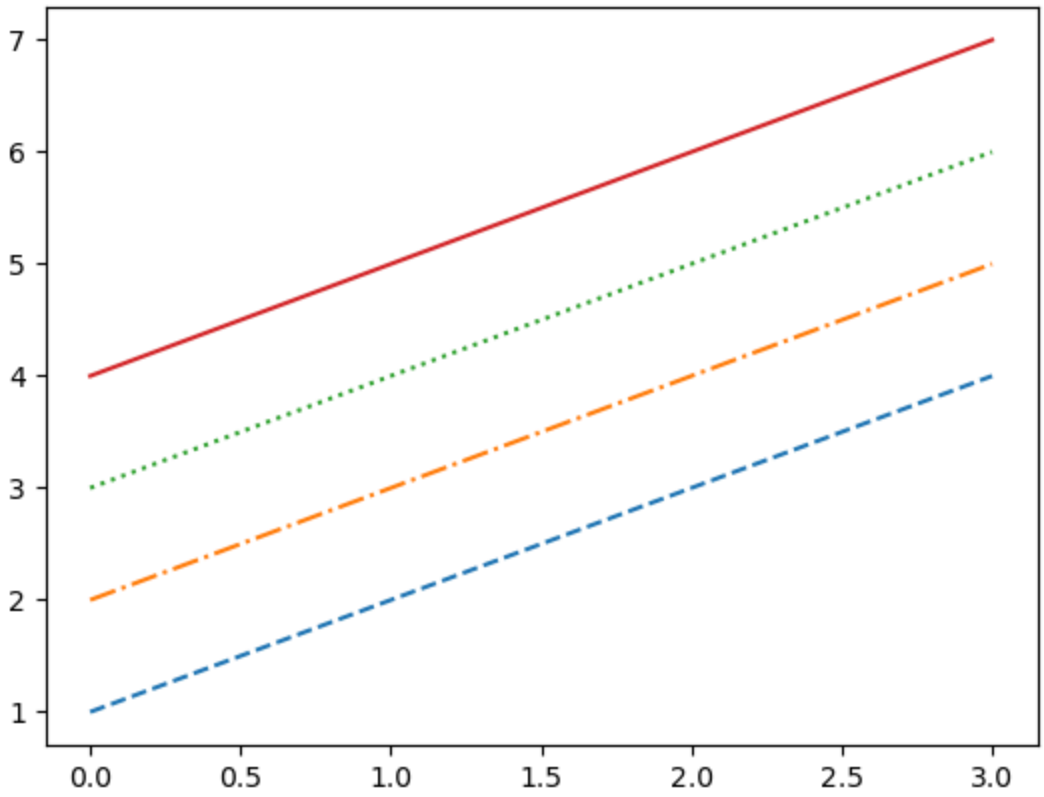
Control colours

In [80]:
```
x16 = np.arange(1, 5)

plt.plot(x16, 'r')
plt.plot(x16+1, 'g')
plt.plot(x16+2, 'b')

plt.show()
```



Control line styles

In [82]:
```
x16 = np.arange(1, 5)

plt.plot(x16, '--', x16+1, '-.', x16+2, ':', x16+3, '-')

plt.show()
```



In [ ]: